

Progress Report 4

Student Name: Eric Sanjo

Student ID: 300395898

Project: Smart Door Anomaly Detection & Cybersecurity for Elderly Care

Course: CSIS 4495 – Applied Research Project, Section 3

Reporting Period: March 10 – March 16, 2026

Report Date: March 16, 2026

Work Date / Hours Logs

Date	Hours	Description of Work Done
Mar 10, 2026	3.0	Downloaded the UR Fall Detection (URFD) dataset (30 fall + 40 ADL videos) and organized it under Implementation/data/urfd/. Implemented Implementation/scripts/extract_keypoints.py using MediaPipe PoseLandmarker (Tasks API) to process all 70 videos and extract 33 pose landmarks (x, y) per frame. Generated ~12,000 frames across 70 CSV files in Implementation/data/keypoints/. Updated .gitignore and Implementation/README.md with Phase 2 LSTM setup instructions.
Mar 11, 2026	3.0	Built Implementation/scripts/train_lstm.py to load keypoint CSVs, normalize using StandardScaler, create 30-frame sequences, and train a two-layer LSTM classifier in TensorFlow/Keras. Trained on URFD (157 fall, 543 normal sequences) achieving ~93% test accuracy. Saved model (fall_lstm.keras) and scaler (fall_lstm_scaler.pkl) and validated using classification report and confusion matrix.
Mar 12, 2026	3.0	Implemented Implementation/models/fall_detection_trained.py with LSTMFallDetector class using MediaPipe for keypoint extraction, rolling frame buffer, scaler application, and LSTM inference. Updated Implementation/scripts/fall_detection_camera.py with --lstm flag to switch between rules-based and LSTM detection while preserving existing features (overlay, FPS, API logging). Tested both modes for real-time performance and accuracy.
Mar 16, 2026	1.5	Standardized environment to Python 3.11 (Homebrew) and resolved TensorFlow/Metal issues for stable local execution. Team meeting

Total Hours: 10.5

Summary Description of Work

This week I extended the fall detection subsystem from a rules-based approach to a complete LSTM-based pipeline trained on the UR Fall Detection (URFD) dataset.

I automated preprocessing by developing `extract_keypoints.py`, which uses MediaPipe to convert raw videos into normalized pose keypoint sequences. On top of this dataset, I implemented and trained an LSTM classifier (`train_lstm.py`) in TensorFlow/Keras, achieving ~93% test accuracy. The trained model and scaler were saved for integration into the system.

To maintain modularity, I introduced a separate `LSTMFallDetector` in `models/fall_detection_trained.py`, leaving the original rules-based detector unchanged. I updated the camera pipeline to support a `--lstm` flag, allowing seamless switching between Phase 1 (rules-based) and Phase 2 (ML-based) detection without modifying external APIs.

The final result is a flexible and extensible fall detection subsystem that supports both deterministic and machine learning approaches, enabling further integration by team members for dashboard visualization and facial recognition enhancements.

Repo Check-in of Implementation

The following files were committed:

- `Implementation/.gitignore` — updated to ignore URFD data, virtual environments, and cache files
- `Implementation/README.md` — added Phase 2 LSTM setup instructions
- `Implementation/scripts/extract_keypoints.py` — keypoint extraction pipeline (MediaPipe)
- `Implementation/data/keypoints/*.csv` — 70 CSV files generated from URFD dataset
- `Implementation/scripts/train_lstm.py` — LSTM training pipeline
- `Implementation/models/fall_lstm.keras` — trained LSTM model
- `Implementation/models/fall_lstm_scaler.pkl` — feature scaler
- `Implementation/models/fall_detection_trained.py` — LSTM-based detector
- `Implementation/scripts/fall_detection_camera.py` — added `--lstm` mode switch
- `Implementation/requirements.txt` — added TensorFlow dependency

AI Use Section

AI Tool	Version / Account	Feature Used	Value Added
GitHub Copilot	Free Tier	Code generation for LSTM layers, training loops, and script refactoring	Verified all generated code, adapted to dataset structure, tuned hyperparameters, and integrated logging and model persistence
ChatGPT	Pro Tier	Research on LSTM architectures, sequence windowing strategies, and fall detection approaches	Adapted concepts to URFD dataset, optimized sequence size (30 frames), and aligned model design with real-time constraints

Appendix: Prompt History

Prompt 1:

“UR Fall Detection dataset, MediaPipe Pose (33 landmarks per frame). How should I convert the videos into fixed-length sequences for an LSTM (window size, overlap, train/test split)?”

Prompt 2:

“With CSVs of 66 pose features per frame + binary label, what is a simple 2-layer LSTM model in TensorFlow/Keras for fall vs normal, and which evaluation metrics should I report?”

Prompt 3:

“I already have a rules-based FallDetector in fall_detection.py. What is a clean way to add a new LSTM-based detector without breaking existing code, and how should I toggle between them from the camera script?”